Rockchip IDC 使用教程

文件标识：RK-KF-SF-049

发布版本：v1.1.1

日期：2026-04-21

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

产品版本

与本文档相对应的产品版本如下：

产品名称	产品版本
RK3588	
RK3576	
RK3572	
RK3562	

修订记录

版本号	作者	修改日期	修改说明
Beta-v0	陈杰克	2024-06-20	初稿
Beta-v1	陈杰克	2025-01-21	修改 Lut 应用输出图尺寸要求。
v1.0.0	陈杰克	2025-04-01	修改默认 OpenCL Program 缓存路径说明。
v1.0.1	陈杰克	2025-10-30	初始化参数支持指定 OpenCL/Mali 库路径。
v1.1.0	陈杰克	2026-02-26	更新 Mesh 表步长描述。
v1.1.1	陈杰克	2026-04-21	支持 RK3572

Rockchip IDC 使用教程

1. 概述
    1.1 版本号说明
    1.2 主要函数列表
2. 应用接口说明
    2.1 LUT
        2.1.1 RKALG_IDC_LUT_GetVersion
        2.1.2 RKALG_IDC_LUT_Init
        2.1.3 RKALG_IDC_LUT_DoLut
        2.1.4 RKALG_IDC_LUT_Deinit
3. 参数定义说明
    3.1 公共参数
        3.1.1 RKALG_IDC_IMG_FMT_E
        3.1.2 RKALG_IDC_MESH_TYPE_E
        3.1.3 RKALG_IDC_IMAGE_S
        3.1.4 RKALG_IDC_MESH_S
    3.2 LUT 参数
        3.2.1 RKALG_LUT_CTX_S
        3.2.2 RKALG_LUT_MODE_E
        3.2.3 RKALG_LUT_OP_E
        3.2.4 RKALG_LUT_INIT_PARAMS_S
        3.2.5 RKALG_LUT_OP_ATTR_S
        3.2.6 RKALG_LUT_TASK_S
4. 函数接口调用示例代码
    4.1 LUT 图像重映射

1. 概述

本文提供 IDC 方案的使用教程，包括应用接口的说明、参数定义和调用示例代码。

IDC 的基本原理、功能介绍，见 "Rockchip_IDC_Developer_Guide_CN.md".

1.1 版本号说明

IDC 内部每个应用模块独立管理版本号。

版本号格式与规则

```
IDC app Version: major.minor.revision.subversion date:YYYY-MM-DD
```

其中 app 为应用模块的名称，如：

```
IDC LUT Version: 1.0.1.66 date: 2025-03-04
```

1.2 主要函数列表

主要的 API 接口如下所示。

函数名称	功能描述
RKALG_IDC_LUT_GetVersion	获取 LUT 应用版本号信息
RKALG_IDC_LUT_Init	LUT 应用初始化，资源申请
RKALG_IDC_LUT_DoLut	LUT 应用执行图像重映射任务
RKALG_IDC_LUT_Deinit	LUT 应用反初始化，资源回收

2. 应用接口说明

本章按应用模块列出 API 说明。

2.1 LUT

2.1.1 RKALG_IDC_LUT_GetVersion

【函数功能】获取 LUT 应用接口的版本号信息。

【函数定义】

```
void RKALG_IDC_LUT_GetVersion(char idcLutVersion[128]);
```

【参数列表】

名称	属性	作用描述
idcLutVersion	output	存放 LUT 接口的版本号信息字符串。

【注意事项】

无。

【返回值】

无。

【用法示例】

```
    char version[128];
    RKALG_IDC_LUT_GetVersion(version);
    printf("%s\n", version);
```

2.1.2 RKALG_IDC_LUT_Init

【函数功能】根据给定参数，初始化 LUT 应用上下文用于后续图像重映射。

【函数定义】

```
int RKALG_IDC_LUT_Init(RKALG_LUT_CTX_S *pLutCtx, RKALG_LUT_INIT_PARAMS_S *pInitParams);
```

【参数列表】

名称	属性	作用描述
pLutCtx	output	LUT 上下文结构体指针
pInitParams	input	LUT 初始化结构体指针

【注意事项】

- 如果没有 OpenCL 程序缓存，首次初始化时间较长。首次初始化以后，OpenCL 程序将缓存到本地，后续初始化过程会显著降低。
- 默认缓存路径为 `/data/`，后续将开放可配置的缓存路径。

【返回值】

0 - 成功；非 0 - 失败。

【用法示例】详见第 4 章。

2.1.3 RKALG_IDC_LUT_DoLut

【函数功能】执行图像重映射任务，一个图像重映射任务包括输入图、输出图、查找表、算子属性。

【函数定义】

```
int RKALG_IDC_LUT_DoLut(RKALG_LUT_CTX_S *pLutCtx, RKALG_LUT_TASK_S *pTask);
```

【参数列表】

名称	属性	作用描述
pLutCtx	input	LUT 上下文结构体指针
pTask	input	LUT 任务结构体指针

【注意事项】

- 如果输入图和输出图的分辨率与初始化时保持一致，则无需重新初始化，直接调用该接口即可。否则需要重新初始化。
- 自动模式下，算子性能统计表的 .csv 文件默认保存在 `/data/`，后续将开放可配置的统计表路径。

【返回值】

0 - 成功；非 0 - 失败。

【用法示例】详见第 4 章。

2.1.4 RKALG_IDC_LUT_Deinit

【函数功能】析构 LUT 上下文并回收相关资源。

【函数定义】

```
int RKALG_IDC_LUT_Deinit(RKALG_LUT_CTX_S *pLutCtx);
```

【参数列表】

名称	属性	作用描述
pLutCtx	input	LUT 上下文结构体指针

【注意事项】

无。

【返回值】

0 - 成功；非 0 - 失败。

【用法示例】详见第 4 章。

3. 参数定义说明

参数包括 IDC 工程内部的公共参数和各应用独立使用的模块参数，本章分别列出介绍。

3.1 公共参数

3.1.1 RKALG_IDC_IMG_FMT_E

【说明】

图像的格式，目前支持 NV12(YUV420SP) 格式的图像。

【定义】

```
typedef enum _rkAlgIDC_IMG_FMT_E {
    RKALG_IDC_IMG_FMT_NV12 = 0,
    RKALG_IDC_IMG_FMT_GRAY = 1
} RKALG_IDC_IMG_FMT_E;
```

【注意事项】

无。

3.1.2 RKALG_IDC_MESH_TYPE_E

【说明】

查找表的类型。查找表的基本概念可参考 "RK IDC 调试指南"。

【定义】

```
typedef enum _rkAlgIDC_MESH_TYPE_E {
    RKALG_IDC_MESH_TYPE_COMMON = 0, /* meshX and meshY, for GPU */
    RKALG_IDC_MESH_TYPE_MERGED = 1, /* meshXy, for GPU */
    RKALG_IDC_MESH_TYPE_FEC    = 2  /* for FEC */
} RKALG_IDC_MESH_TYPE_E;
```

【注意事项】

- 目前仅支持使用合并浮点类型（即`RKALG_IDC_MESH_TYPE_MERGED`）的查找表。

3.1.3 RKALG_IDC_IMAGE_S

【说明】

定义单帧图像信息的结构体。

【定义】

```
typedef struct _rkAlgIDC_IMAGE_S {
    uint64_t id;
    RKALG_IDC_IMG_FMT_E eImgFmt;
    uint32_t u32Width;
    uint32_t u32Height;
    uint32_t u32Stride[3];
    uint32_t u32HgtStride[3];
    int fd[3];
    void *virAddr[3];
    void *phyAddr[3];
} RKALG_IDC_IMAGE_S;
```

【成员】

名称	描述
id	图像帧的标识符，外部调用时使用。
eImgFmt	图像格式。
u32Width	图像宽度（实宽）。
u32Height	图像高度（实高）。
u32Stride	图像宽跨距（虚宽）。最多为 3 个通道：Y, U, V 各占一个通道。
u32HgtStride	图像高跨距（虚高）。最多为 3 个通道。
fd	DRM 设备文件描述符。最多为 3 个通道。
virAddr	图像数据虚拟地址。最多为 3 个通道。
phyAddr	图像数据物理地址。最多为 3 个通道。

【注意事项】

- 物理地址为预留数据段，暂不支持。
- 3 通道的数据段是为 Y, U, V 预留，配置方式参考每个应用模块的接口和参数说明。

3.1.4 RKALG_IDC_MESH_S

【说明】

定义查找表信息的结构体。

【定义】

```
typedef struct _rkAlgIDC_MESH_S {
    uint64_t id;
    uint32_t u32StepX;
    uint32_t u32StepY;
    uint32_t u32Width;
    uint32_t u32Height;
    uint32_t u32Stride;
    uint32_t u32HgtStride;
    RKALG_IDC_MESH_TYPE_E eMeshType;
    int fd[4];
    void *virAddr[4];
    void *phyAddr[4];
} RKALG_IDC_MESH_S;
```

【成员】

名称	描述
id	图像帧的标识符，外部调用时使用。
u32StepX	查找表在目标图像 x 方向上的采样步长，单位：pixel。
u32StepY	查找表在目标图像 y 方向上的采样步长，单位：pixel。
u32Width	查找表宽度，即 x 方向上的采样点数。
u32Height	查找表高度，即 y 方向上的采样点数。
u32Stride	查找表宽跨距。
u32HgtStride	查找表高跨距。
eMeshType	查找表类型。
fd	DRM 设备文件描述符。最多为 4 个通道。
virAddr	查找表数据虚拟地址。最多为 4 个通道。
phyAddr	查找表数据物理地址。最多为 4 个通道。

【注意事项】

- 物理地址为预留数据段，暂不支持。
- 多通道的数据段的配置方式，参考每个应用模块的接口和参数说明。
- 采样步长需设为 2 的幂次且最小为 4。
- 采样点数的和步长、图像宽高满足需以下关系：

```
X 方向采样点数 width  >= (dstWidth-1)  / StepX + 2;
Y 方向采样点数 height >= (dstHeight-1) / StepY + 2;
```

3.2 LUT 参数

3.2.1 RKALG_LUT_CTX_S

【说明】

LUT 应用上下文指针结构体。

【定义】

```
typedef struct _rkAlgLutCtx_S {
    void *p;
} RKALG_LUT_CTX_S;
```

【成员】

名称	描述
p	算法内部上下文指针。

【注意事项】

无。

3.2.2 RKALG_LUT_MODE_E

【说明】

枚举 LUT 应用模式。

【定义】

```
typedef enum _rkAlgLutMode_E {
    RKALG_IDC_LUT_DEFAULT_MODE = 0, /* Use default Kernel and NDRange */
    RKALG_IDC_LUT_MANUAL_MODE  = 1, /* Use user defined Kernel and NDRange */
    RKALG_IDC_LUT_AUTO_MODE    = 2  /* Auto Detect best Kernel and NDRange. This takes some
extra time at startup*/
} RKALG_LUT_MODE_E;
```

【注意事项】

无。

3.2.3 RKALG_LUT_OP_E

【说明】

枚举 LUT 重映射算子。

【定义】

```
typedef enum _rkAlgLutOp_E {
    RKALG_IDC_LUT_OP_0 = 0, /* default op*/
    RKALG_IDC_LUT_OP_1 = 1,
    RKALG_IDC_LUT_OP_2 = 2,
    RKALG_IDC_LUT_OP_3 = 3
} RKALG_LUT_OP_E;
```

【注意事项】

无。

3.2.4 RKALG_LUT_INIT_PARAMS_S

【说明】

LUT 应用初始化参数结构体。

【定义】

```
typedef struct _rkAlgLutInitParams_S {
    uint32_t u32SrcWidth;     /* 2 pixel aligned */
    uint32_t u32SrcHeight;
    uint32_t u32SrcStride;    /* 64 pixel aligned */
    uint32_t u32SrcHgtStride; /* 2 pixel aligned */
    uint32_t u32DstWidth;
    uint32_t u32DstHeight;    /* 4 pixel aligned */
    uint32_t u32DstStride;    /* 16 pixel aligned */
    uint32_t u32DstHgtStride; /* 8 pixel aligned */
    uint32_t u32Affinity;
    char     cOpenclLibraryPath[256]; /* (optional) OpenCL/mali Library Path. If missing, the
default system library path will be used. */
    RKALG_LUT_MODE_E eMode;
} RKALG_LUT_INIT_PARAMS_S;
```

【成员】

名称	描述
u32SrcWidth	输入图像宽。
u32SrcHeight	输入图像高。
u32SrcStride	输入图像宽跨距。
u32SrcHgtStride	输入图像高跨距。
u32DstWidth	输出图像宽。
u32DstHeight	输出图像高。
u32DstStride	输出图像宽跨距。
u32DstHgtStride	输出图像高跨距。
u32Affinity	CPU 亲和性参数。
cOpenclLibraryPath	指定 opencl/mali library，动态检索和链接时会优先选择。 若此字段为空，则使用默认系统库路径。
eMode	LUT 应用模式。

【注意事项】

- 初始化信息需符合以下对齐要求。 输入图：虚宽 64 像素对齐，虚高 2 像素对齐。 输出图：虚宽 16 像素对齐，虚高 8 像素对齐。
- 如果设置为手动模式，后续调用执行 API 时，需要配置算子属性数据段。

3.2.5 RKALG_LUT_OP_ATTR_S

【说明】

定义重映射算子属性。

【定义】

```
typedef struct _rkAlgLutOpAttr_S {
    RKALG_LUT_OP_E eOp;       /* Only work in manual mode. */
    uint32_t u32LocalSize[2]; /* Only work in manual mode. */
} RKALG_LUT_OP_ATTR_S;
```

【成员】

名称	描述
eOp	重映射算子类型。
u32LocalSize	协同执行的线程规模。

【注意事项】

- 手动配置算子属性时，重映射算子类型为必填项，localSize 为选填项，如省略则由算法内部自动计算，不一定效率最优。

3.2.6 RKALG_LUT_TASK_S

【说明】

定义重映射任务。一个重映射任务的要素包括：输入图、输出图、查找表，手动模式下还包括算子属性。

【定义】

```
typedef struct _rkAlgLutTask_S {
    RKALG_IDC_IMAGE_S   *pSrcImage; /* src attr and data */
    RKALG_IDC_IMAGE_S   *pDstImage; /* dst attr and data */
    RKALG_IDC_MESH_S    *pMesh;     /* mesh attr and data */
    RKALG_LUT_OP_ATTR_S *pOpAttr;   /* op attr, only work in manual mode.*/
} RKALG_LUT_TASK_S;
```

【成员】

名称	描述
pSrcImage	输入图像帧指针。
pDstImage	输出图像帧指针。
pMesh	查找表结构体指针。
pOpAttr	算子属性结构体指针。仅手动模式下需要设置。

【注意事项】

- 查找表
  - 类型要求：目前仅支持使用 GPU 处理合并浮点类型的查找表，表中 X 和 Y 坐标需为交替存储的形式。
  - 多通道数据段配置：数据地址、fd，只需配置首通道。
- 图像帧
  - 格式要求：目前仅支持 YUV420SP 格式的图像，需按以下要求对齐。 输入图：虚宽 64 像素对齐，虚高 2 像素对齐。 输出图：虚宽 16 像素对齐，虚高 8 像素对齐。
  - 多通道数据段配置：虚宽、虚高、数据地址、fd，只需配置首通道。

4. 函数接口调用示例代码

4.1 LUT 图像重映射

LUT 应用实现图像重映射功能，根据查找表在输入图上进行查表计算，得到输出图。

LUT 应用支持默认模式、手动模式、自动模式，在效率上有所区别。以下展示默认模式下调用 API 完成图像重映射的代码。

【注意事项】

- 需要事先准备好合并浮点类型的查找表，如果只有其他类型的查找表，需先将其改写为合并浮点的类型。
- 如果任务分辨率不变，支持查找表动态更新，无需重新初始化。
- 可通过 `RKALG_IDC_LOG_LEVEL` 环境变量更改日志等级，范围为[0, 7]（默认为 4），从小到大依次打印更多内容，如：

  ```
  export RKALG_IDC_LOG_LEVEL=5
  ```

【示例代码】

```
void test()
{
    int ret = 0;
    printf("test begin.\n");

    char version[128];
    RKALG_IDC_LUT_GetVersion(version);
    printf("%s\n", version);

    /* Step 1-1. Config initial params */
    RKALG_LUT_INIT_PARAMS_S stInitParams;
    stInitParams.u32SrcWidth  = 1920;
    stInitParams.u32SrcHeight = 1080;
    stInitParams.u32SrcStride = alignUp(stInitParams.u32SrcWidth, 64);
    stInitParams.u32SrcHgtStride = alignUp(stInitParams.u32SrcHeight, 2);
    stInitParams.u32DstWidth  = 1920;
    stInitParams.u32DstHeight = 1080;
    stInitParams.u32DstStride = alignUp(stInitParams.u32DstWidth, 16);
    stInitParams.u32DstHgtStride = alignUp(stInitParams.u32DstHeight, 8);
    stInitParams.eMode        = RKALG_IDC_LUT_DEFAULT_MODE;
    sprintf(stInitParams.cOpenclLibraryPath, "/usr/lib/libmali.so");

    char dst_img_path[256];
    char src_img_path[256];
    sprintf(src_img_path, TMP_DATA_PATH"input_%dx%d_nv12.yuv", stInitParams.u32SrcWidth,
stInitParams.u32SrcHeight);
```

```
    sprintf(dst_img_path, TMP_DATA_PATH"output_%dx%d_nv12.yuv", stInitParams.u32DstWidth,
stInitParams.u32DstHeight);
    char meshxy_path[] = TMP_DATA_PATH"/rkalg_idc_gpu_meshxy00.bin";

    /* Step 1-2. Create LUT context */
    RKALG_LUT_CTX_S stLutCtx;
    RKALG_IDC_LUT_Init(&stLutCtx, &stInitParams);

    /* Step 2-1. malloc DMA Buffer for srcImg and read data */
    RKALG_IDC_IMAGE_S srcImage;
    memset(&srcImage, 0, sizeof(RKALG_IDC_IMAGE_S));
    srcImage.eImgFmt         = RKALG_IDC_IMG_FMT_NV12; // only NV12 supported now
    srcImage.u32Width        = stInitParams.u32SrcWidth;
    srcImage.u32Height       = stInitParams.u32SrcHeight;
    srcImage.u32Stride[0]    = stInitParams.u32SrcStride;
    srcImage.u32HgtStride[0] = stInitParams.u32SrcHgtStride;

    DmaBufferInfo dmaBufSrc;
    memset(&dmaBufSrc, 0, sizeof(DmaBufferInfo));
    int bpp_nv12 = 12 * sizeof(unsigned char);
    alloc_drm_buf(&dmaBufSrc, srcImage.u32Stride[0], srcImage.u32HgtStride[0], bpp_nv12);
    srcImage.fd[0]      = dmaBufSrc.fd;
    srcImage.virAddr[0] = dmaBufSrc.ptr;

    size_t srcImgSize = srcImage.u32Width * srcImage.u32Height * 3 / 2 * sizeof(unsigned
char);
    ret |= readBinaryFile(src_img_path, srcImage.virAddr[0], srcImgSize);
    if (ret != 0) {
        printf("read src image failed.\n");
        return;
    }

    /* Step 2-2. malloc DMA Buffer for dstImg and config */
    RKALG_IDC_IMAGE_S dstImage;
    memset(&dstImage, 0, sizeof(RKALG_IDC_IMAGE_S));
    dstImage.eImgFmt         = RKALG_IDC_IMG_FMT_NV12; // Only support NV12 now
    dstImage.u32Width        = stInitParams.u32DstWidth;
    dstImage.u32Height       = stInitParams.u32DstHeight;
    dstImage.u32Stride[0]    = stInitParams.u32DstStride;
    dstImage.u32HgtStride[0] = stInitParams.u32DstHgtStride;

    DmaBufferInfo dmaBufDst;
    memset(&dmaBufDst, 0, sizeof(DmaBufferInfo));
    alloc_drm_buf(&dmaBufDst, dstImage.u32Stride[0], dstImage.u32HgtStride[0], bpp_nv12);
    dstImage.fd[0]      = dmaBufDst.fd;
    dstImage.virAddr[0] = dmaBufDst.ptr;

    /* Step 2-3. malloc DMA Buffer for Xy mesh and read data */
    RKALG_IDC_MESH_S mesh;
    memset(&mesh, 0, sizeof(RKALG_IDC_MESH_S));
    mesh.u32StepX     = 32;
    mesh.u32StepY     = 16;
    mesh.u32Width     = (dstImage.u32Stride[0] - 1) / mesh.u32StepX + 2;
```

```
    mesh.u32Height    = (dstImage.u32Height - 1) / mesh.u32StepY + 2;
    mesh.u32Stride    = mesh.u32Width;
    mesh.u32HgtStride = mesh.u32Height;
    mesh.eMeshType    = RKALG_IDC_MESH_TYPE_MERGED;

    DmaBufferInfo dmaBufMesh;
    memset(&dmaBufMesh, 0, sizeof(DmaBufferInfo));
    size_t bpp_mesh_merged = 8 * sizeof(float) * 2;
    alloc_drm_buf(&dmaBufMesh, mesh.u32Width, mesh.u32Height, bpp_mesh_merged);
    mesh.fd[0]      = dmaBufMesh.fd;
    mesh.virAddr[0] = dmaBufMesh.ptr;
    size_t meshSize = mesh.u32Width * mesh.u32Height * sizeof(float) * 2;
    ret |= readBinaryFile(meshxy_path, mesh.virAddr[0], meshSize);
    if (ret != 0) {
        printf("read mesh data failed.\n");
        return ;
    }

    /* Step 2-4. Config Op Attr. */
    RKALG_LUT_OP_ATTR_S stOpAttr;
    memset(&stOpAttr, 0, sizeof(RKALG_LUT_OP_ATTR_S));

    /* Step 2-5. Call doLut API */
    RKALG_LUT_TASK_S stTask;
    stTask.pSrcImage = &srcImage;
    stTask.pDstImage = &dstImage;
    stTask.pMesh     = &mesh;
    stTask.pOpAttr   = &stOpAttr; // only take effect when mode is
RKALG_IDC_LUT_MANUAL_MODE.
    RKALG_IDC_LUT_DoLut(&stLutCtx, &stTask);

    /* Step 2-6. Write dstImg to file */
    FILE *fp_out = fopen(dst_img_path, "wb");
    if (fp_out == NULL) {
        printf("Can not open %s\n", dst_img_path);
        return;
    }
    unsigned char* pt     = NULL;
    unsigned char* pDstY  = (unsigned char*)dmaBufDst.ptr;
    unsigned char* pDstUV = pDstY + dstImage.u32Stride[0] * dstImage.u32HgtStride[0];

    for (size_t row_idx = 0; row_idx < dstImage.u32Height; row_idx++) {
        pt = pDstY + row_idx * dstImage.u32Stride[0];
        fwrite(pt, sizeof(unsigned char), dstImage.u32Width, fp_out);
    }
    for (size_t row_idx = 0; row_idx < dstImage.u32Height / 2; row_idx++) {
        pt = pDstUV + row_idx * dstImage.u32Stride[0];
        fwrite(pt, sizeof(unsigned char), dstImage.u32Width, fp_out);
    }
    fclose(fp_out);
    printf("write %s success.\n", dst_img_path);

    /* Step 3. Call deInit API */
```

```
    RKALG_IDC_LUT_Deinit(&stLutCtx);
    if (dmaBufMesh.ptr != nullptr) {
        drm_munmap(dmaBufMesh.ptr, dmaBufMesh.size);
        dmaBufMesh.ptr = nullptr;
    }
    close(dmaBufMesh.fd);
    if (dmaBufSrc.ptr != nullptr) {
        drm_munmap(dmaBufSrc.ptr, dmaBufSrc.size);
        dmaBufSrc.ptr = nullptr;
    }
    close(dmaBufSrc.fd);
    if (dmaBufDst.ptr != nullptr) {
        drm_munmap(dmaBufDst.ptr, dmaBufDst.size);
        dmaBufDst.ptr = nullptr;
    }
    close(dmaBufDst.fd);

    printf("test end.\n");
}
```